PennyPal — TECHWIZ 7

Presentation & Viva Script

Team: TechBeast | **Center:** Aptech North Nazimabad | **Category:** Multi-Platform App Computing
Project tagline: Make Every Penny Count.

Part 1 — Opening & Team Introduction

Good morning, everyone! First of all, thank you so much for giving us this opportunity to present our project. My name is Muhammad Ammad Anees, and I'm the team lead of Team TechBeast from Aptech North Nazimabad Center.

Before we get started, let me quickly introduce my team members who are here with me today: so, we have Muhammad Hadi, who worked on the UI design; Ahmed Raza, who worked on the admin panel; and Abdul Qadir, who handled the project documentation.

Together, we developed an application called **PennyPal**, with one simple goal: to help students manage their money better. Our motto is: **Make Every Penny Count.** Today, I'll explain why we built PennyPal, how it works, and then I will share my screen and demonstrate all its main features live. So, let's get started!

Part 2 — The Problem We're Solving

Now, let me give you a little background about why we chose this project. You know, in TECHWIZ, the management provided us with three project options, and our team picked PennyPal without hesitation. Because as students, this is a real problem we experience every single day.

Students have different sources of income. Some receive pocket money from home, some earn through freelancing or part-time jobs, and others receive scholarships or stipends. **But regardless** of where the money comes from, keeping track of daily spending is really difficult. **You spend** a little bit on food, a little on transport, maybe pay for a subscription... and basically, by the third week of the month, you wonder: **"Where did all my money go?"**

We wanted to make managing money simple and stress-free. **That's why we chose** PennyPal. **It helps students** record their daily income and expenses, set monthly budgets, track their savings, and understand their spending habits. And most importantly, we wanted the app to work even when the internet is unavailable, because in campus classrooms and basements, students don't always have a reliable connection. So, **our goal** was to create a simple, useful financial companion for student life.

Part 3 — Technologies We Used

Now, before we jump into the live demo, let me briefly explain how we built PennyPal. Following SRS Section 1.8, we chose a modern, scalable stack:

- **Flutter:** builds our application for mobile, tablet, and web from a single codebase.
- **Riverpod:** manages application state so screens update instantly without lag.
- **Firebase Firestore:** stores our data with automatic offline caching for zero-internet use.
- **Google Gemini AI:** reads paper receipts and auto-fills expense forms.
- **Cloudinary & Brevo:** handle cloud media storage and transactional email alerts.

In simple words: fast, secure, and completely offline-ready. Alright, now let me share my screen and show you PennyPal live in action!

Part 4 — Live Application Demonstration

Step 1: Welcome & Onboarding Flow

Action: Show the onboarding screen, swipe across the introductory slides, point to the 4 pillars, then tap "I already have an account" or "Skip" to reach Login.

Alright, so when a student opens PennyPal for the very first time, they are welcomed by this clean onboarding flow.

As outlined in SRS Section 1.4, it introduces our four core pillars: tracking daily expenses, setting monthly budgets, saving for goals, and building smart money habits.

From here, a new student can tap **Create account**, or if they already have an account, tap **I already have an account** right here... which takes us straight to Login.

Step 2: Login & Authentication

Action: Show the login screen, enter demo credentials, and tap Sign In.

Now, right here at the login screen, we have secure authentication to protect student privacy. PennyPal also has a separate, protected area for administrators.

Let me enter our student credentials... and tap **Sign In**. And just like that, the app verifies the account against Firebase and takes us straight into the main student dashboard!

Step 3: Dashboard (The Financial Command Center)

Action: Point to top header (profile picture, greeting, notification bell), then Net Balance hero, Income/Expense cards, central Pace Ring, and Cash Runway.

Now, here we are on the student dashboard, which serves as the main command center of PennyPal.

Right at the top header, you can see the student profile picture, personalized greeting, and real-time notification bell for budget alerts.

Directly below is our main financial command center: First, we have our **Net Balance** hero card, along with Total Income and Total Expenses for the month. You know, students don't keep all their money in one place—some is physical cash in pocket, some in a bank card, and some in mobile wallets like Nayapay or EasyPaisa. PennyPal tracks each payment mode separately, while combining them into this one true net balance.

Now, take a closer look at this **Budget Pace Ring** right in the center: it visually calculates the student's daily spending speed in real time. If spending is healthy, it stays green. At 80 percent, it warns in amber, and if over budget, it turns coral red.

Right below that, we have the **Financial Health Score** and **Cash Runway Days**. It actually calculates: based on your current spending, your remaining money will last for, say, 18 more days. This gives students an immediate reality check so they don't run out of money before the month ends.

And down here, the **Quick Category Ribbon** and **Recent Transactions** let students review their latest spending at a glance. So from this one single screen, a student gets complete clarity over their entire financial situation.

Step 4: Monthly Budgets

Action: Tap the Budget section, point to category limits, then show the 1-Tap Quick Copy banner.

Now, seeing your balance is good, but controlling spending is what really matters. So let's open the **Budget** section. Here, students can set spending limits for different categories, like food, transport, and education. And you know, student allowances and rent run on a monthly basis, so PennyPal organizes budgets strictly by calendar month.

Now, here is a really neat feature we built, called **1-Tap Quick Copy**. You see, when a new month begins, students don't want to type every single category limit all over again. With just one tap right here, PennyPal automatically copies last month's budget. So, this saves time and makes monthly planning super easy.

Step 5: Add an Expense and Scan a Receipt

Action: Tap Add Expense (+), show the 8 categories, then tap Attach Receipt / Scan.

Now, once your budget is set, you need to record your daily spending. So let me tap this plus button to add an expense. As you can see, we have all 8 standard categories ready—like Food, Transport, and Bills. Normally, you can just type the amount and details manually.

But check this out: if a student has a paper receipt from the canteen or a bookstore, they don't even have to type! Let me tap **Scan Receipt**. Our Gemini AI reads the image, extracts the merchant name, the date, and the total amount, and fills in the form automatically. So, all the student has to do is review the information and tap **Save**.

Step 6: Offline Mode (Key Feature Demonstration)

Action: Pull down quick settings, turn ON Airplane Mode, add a test expense, tap Save, show it appearing, then turn OFF Airplane Mode.

Now, I want to show you one of the most important features of PennyPal: our offline engine. Following the offline requirement of SRS Section 1.5, let me demonstrate what happens:

Watch this—I'll turn on **Airplane Mode** right now. The phone has zero internet. Now, let's add a quick 15-dollar expense for campus lunch... tap Save... and look at that! As you can see, the expense appears in the application immediately. There is no waiting, no loading spinner, and no error.

And now, watch what happens when I turn Airplane Mode back off: as soon as the connection returns, Firestore automatically syncs the pending transaction with the cloud in the background. So, this helps students continue recording their expenses anytime, anywhere.

Step 7: Savings Goals

Action: Tap the Savings tab, open an active goal (like 'New Laptop'), and tap Contribute to add $20.

Now, tracking daily expenses is important, but building good money habits also means saving for the future. So let's switch over to **Savings Goals**. You see, students often want to save for something big—for example, buying a laptop or paying for a professional certification. They can set a target amount, choose a deadline date, and track their progress.

Let me show you: I'll open this laptop goal, and let's add a 20-dollar contribution right now... Notice how the progress bar moves forward immediately, the app recalculates the remaining timeline, and it gives a motivational milestone to keep students encouraged.

Step 8: Extra Student Features

Action: From dashboard shortcuts, show Split Bill, Borrow & Lend, and Subscriptions.

Now, under the SRS Section 1.9 innovation clause, we asked our classmates: "*What other money headaches do you face?*" And based on their feedback, we included three practical student tools:

- First, **Split Bill**: calculates each person's share when eating out, and sends a WhatsApp reminder in one tap.
- Second, **Borrow & Lend**: tracks small loans between classmates so friendships stay stress-free.
- And third, **Subscriptions Manager**: tracks recurring monthly fees so allowances aren't silently drained.

Step 9: Learning Hub and AI Assistant

Action: Open a lesson in the Learning Hub, then show the AI Assistant.

We also believe managing money is not just about recording numbers. It's about learning how to make better financial decisions. That's why we included a **Learning Hub** with quick, bite-sized lessons on topics like needs versus wants and emergency savings.

And right next to it, students can also open our **AI Assistant** and ask practical questions—like: *"How can I save 30 dollars this week?"*—and get instant, personalized guidance. Together, these features help students build lasting financial discipline.

Step 10: Admin Panel

Action: Sign out of the student account and sign in to the prepared demo admin account (admin@pennypal.com).

And finally, let me show you the administrative side. Following the role-separation requirement in SRS Section 1.6, administrators get a separate dedicated dashboard. From here, the admin can view registered students, monitor system analytics, respond to support tickets, and even publish new educational lessons using the built-in CMS. And that completes our live demonstration of PennyPal!

Part 5 — Closing Statement

So, to summarize—PennyPal is much more than just an expense tracker. As outlined in the TECHWIZ SRS, it gives students a complete financial companion.

From offline budgeting to smart receipt scanning, our goal was to make a simple, useful app for campus life. As students, we are proud of what we built, and we learned so much from this project.

On behalf of Team TechBeast and Aptech North Nazimabad Center, thank you so much, respected judges, for your time today. We are now ready to answer your questions!

Part 6 — Technical Viva: Simple Answers

Understand the meaning of each answer. Deliver clear, honest answers in simple words.

Q1. Why did you choose this project?
We chose PennyPal because managing money is a common challenge for students. We wanted to build something practical that helps them track expenses, plan budgets, and save money.

Q2. What makes PennyPal different from other finance apps?
PennyPal combines expense tracking and budgeting with offline support, receipt scanning, savings goals, financial lessons, and student-focused features such as bill splitting and debt tracking.

Q3. Why did you use Flutter?
Flutter lets us build applications for multiple platforms using one main codebase. This helps reduce duplicated development work.

Q4. What is Firebase Firestore?
Firestore is a cloud database from Firebase. It stores our application data and allows the app to receive updates when connected to the internet.

Q5. What is Riverpod, and why did you use it?
Riverpod helps us manage application state. It makes it easier for screens and other parts of the app to access data and respond to changes.

Q6. How does offline mode work?
Firestore supports a local cache. When offline support is enabled and the relevant data is available locally, users can continue working, and pending changes can synchronize after the connection returns.

Q7. Does every feature work offline?
Not necessarily. Features that depend on an online service, such as Gemini AI receipt scanning, generally need an internet connection. Standard logging, budgeting, and dashboard calculations work offline.

Q8. How does AI receipt scanning work?
The user selects or captures a receipt image. We send it to Gemini Vision AI, which attempts to identify information such as the merchant, date, and total amount. The extracted information is used to fill the expense form.

Q9. What happens if the AI service is unavailable?
If Gemini AI is offline or the connection times out, our app uses a built-in text pattern fallback. It scans the receipt on the device and looks for numbers and keywords to extract the total amount without crashing.

Q10. How do you protect users' financial information?
We use Firebase Authentication and Firestore security rules to control access. Only authenticated students can access their own accounts. We also never store sensitive card details like CVVs and full card numbers.

Q11. How do you prevent users from accessing each other's data?
We secure user data using Firestore Security Rules. Every document is saved under the student's unique User ID (`request.auth.uid`). This guarantees that one user can never read or edit another user's private financial records.

Q12. How does monthly budgeting work?
Users set spending limits for categories in a particular month. PennyPal compares recorded expenses against those limits so users can understand their spending and remaining budget.

Q13. How does the savings feature help students?
Users can set a savings target and record contributions. They can then track how much they have saved and how much remains to reach their goal.

Q14. Why did you add Split Bill and Borrow & Lend?
Students often share expenses with friends or lend each other small amounts. These features help them calculate shared costs and keep track of money owed.

Q15. What was your biggest challenge?
One of our main challenges was making sure the offline engine worked smoothly. We had to ensure that when a student logs an expense without internet, it saves locally right away and syncs to Firebase automatically once the connection is back.

Q16. How did you test the application?
We tested the app thoroughly: first, manual testing across all screens and user flows, especially Airplane Mode. Second, we wrote automated unit tests for our calculation logic to make sure all financial math and budget percentages are completely accurate.

Q17. Why did you create an admin panel?
The admin panel provides a separate area for administrative tasks, such as viewing system information, handling support requests, and managing educational content.

Q18. Can PennyPal support more users in the future?
It can be developed further using cloud infrastructure, but growth would require testing database usage, security, performance, and operating costs. We would measure those areas before claiming a particular scale.

Q19. What would you improve in the future?
We could improve the application through more testing, better financial insights, additional languages, more accessibility options, and carefully designed features based on student feedback.

Q20. Why should students use PennyPal?
Because it brings their expenses, budgets, and savings goals together in one place. Our aim is to make managing money easier and help students build better financial habits.

Q21. Why did you use Riverpod instead of Provider or setState?
Riverpod catches errors early while writing code and updates only the necessary parts of the screen. This keeps our app fast and prevents crashes.

Q22. What happens to budgets when a new month begins?
Monthly expenses reset to zero for the new month, but the student's all-time net balance carries forward safely.

Q23. How does the app format dates and currencies?
We use Flutter's `intl` package. It formats amounts like `$25.00` and displays clear dates like `Oct 2026`.

Q24. How does the Split Bill feature share to WhatsApp?
We use the `share_plus` package. It formats each student's share into a friendly text message and opens WhatsApp in one tap.

Q25. How do students take pictures of receipts?
We use the `image_picker` package. Students can use their phone camera or pick an existing receipt image from their gallery.

Q26. Does PennyPal connect directly to bank accounts?
No. Following competition guidelines, PennyPal is a personal expense companion, not a banking app. We never ask for bank passwords or card CVVs.

Q27. How does the app alert students about overspending?
We use `flutter_local_notifications`. The app sends instant phone notifications when spending reaches 80 percent or 100 percent of a budget.

Q28. How can a student export their monthly reports?
We use the `pdf` and `printing` packages. Students can generate a clean monthly summary and download or print it directly.

Q29. What happens if the phone loses internet while taking a receipt photo?
The photo stays safely on the device. When internet is back, the student can scan it or save it locally.

Q30. How did your team divide the work?
Muhammad Ammad led the project, Muhammad Hadi designed the UI, Ahmed Raza built the admin panel, and Abdul Qadir handled documentation, all using GitHub.

Part 7 — What to Say When You Need Help

You do not need to panic if a judge speaks quickly or asks something unexpected. Use these phrases naturally:

- **If you didn't hear the question:** "Sorry, could you please repeat the question?"
- **If the question is too fast:** "Could you please speak a little more slowly?"
- **If you need time to think:** "That's a good question. Let me think for a moment."
- **If you understand only part of the question:** "Do you mean how we store the data, or how we keep it secure?"
- **If you don't know the answer:** "I haven't explored that part in enough detail yet, so I don't want to give you incorrect information. I will investigate it further."
- **If a teammate knows the answer better:** "My teammate worked more closely on that part. May I ask him to explain the implementation?"
- **If the judge points to a teammate:** Let them answer. Don't interrupt or try to answer on their behalf unless the judge asks you to add something.

Part 8 — Final Preparation Checklist

- Practise the opening until you can say it comfortably without reading.
- Make sure all four team members can introduce their contributions clearly.
- Prepare the application, test accounts, and sample data before the session.
- Test the offline demonstration in advance.
- Check which features require an internet connection.
- Confirm the admin panel and all demo features work as expected.
- Never display passwords or API keys during screen sharing.
- Be ready to explain any technical term you use in simple words.
- If you don't know something, be honest and explain how you would investigate it.

Part 9 — 1-Line Tech Stack Cheatsheet (Tools & Technologies Used)

Quick, simple 1-line answers if judges ask about any specific tool or service:

- **Flutter:** Cross-platform framework by Google that builds our application for Android, iOS, Web, and Desktop from a single Dart codebase.
- **Firebase Authentication:** Handles secure student registration, email/password login, password resets, and user sessions.
- **Cloud Firestore:** Scalable NoSQL cloud database with automatic offline caching so students can log expenses with zero internet.
- **Cloudinary:** Cloud media service used to store, optimize, and quickly deliver receipt photos and user profile pictures without loading the database.
- **Brevo (formerly Sendinblue):** Transactional email delivery service used for sending account verification emails, password reset links, and budget alerts.
- **Google Gemini AI (Vision & Flash):** Multimodal AI model that reads paper receipt photos, auto-fills transaction details, and answers student budgeting questions in the AI Assistant.
- **Riverpod 2.0:** Reactive, compile-time safe state management library that automatically updates UI screens whenever financial data changes.
- **GoRouter:** Declarative URL router with route guards that keeps user sessions safe and cleanly separates student screens from the admin portal.
- **SharedPreferences:** Lightweight on-device key-value storage used to remember user preferences like dark/light theme, default currency, and onboarding status.
- **Flutter Local Notifications:** Device notification service that sends immediate alerts when spending hits 80% or 100% of a monthly category budget.
- **Pdf & Printing Package:** Standard Flutter libraries used to generate, preview, and download monthly student expense reports in formatted PDF and CSV.
- `intl` **Package:** Formats currencies like `$15.00` and displays clean dates like `Oct 2026` across all transaction screens.
- `image_picker` **Package:** Allows students to take photos of paper receipts using their phone camera or select images from gallery.
- `share_plus` **Package:** Powers the 1-tap WhatsApp sharing in Split Bill so students can send calculated bill shares to friends.
- `device_preview` **Package:** Allows testing and presenting mobile screens on any screen size or browser during live demos.
- `path_provider` **Package:** Finds safe device storage folders to save exported PDF reports before opening or sharing.
- **GitHub:** Cloud Git platform used by our four-member team for version control, code commits, and project collaboration throughout TechWiz.